

Generic Mini Project Problem Statement

Title: ABC Domain Database Management System

Overview

This capstone project simulates a real-world enterprise database system for a specific domain such as Healthcare, Banking, Hospitality, Events, Real Estate, or Ticketing.

Learners are required to design and implement a fully normalized relational database using Microsoft SQL Server. The project emphasizes core database concepts including DDL, DML, DCL, TCL, joins, subqueries, functions, triggers, views, indexes, and normalization.

The project follows an Agile-based execution model with structured deliverables across three sprints.

Project Objective

To design and implement a scalable, normalized, and secure database system that supports domain-specific operations, ensures data integrity, and enables efficient querying and reporting using SQL Server.

Functional Requirements (Applicable to All Domains)

- Master data management for core entities
- Transaction handling based on domain use cases
- Relationship mapping using primary and foreign keys
- Data validation using constraints (NOT NULL, CHECK, UNIQUE)
- Querying using joins, subqueries, and aggregation
- Views for reporting and abstraction
- Indexes for performance optimization
- Triggers for automation and audit logging
- Access control using GRANT and REVOKE
- Transaction handling using COMMIT and ROLLBACK

Technology Stack

- Database: Microsoft SQL Server
 - Query Language: T-SQL
 - Tool: SQL Server Management Studio (SSMS)
 - Version Control: GitHub
 - ERD Tool: Draw.io / Visual Paradigm
-

Sprint-Based Execution Plan

Sprint 1: Database Design and Normalization

- Identify entities and attributes
- Apply normalization up to 3NF/BCNF
- Define relationships
- Create ER Diagram

Deliverables:

- ER Diagram
 - Normalization report
-

Sprint 2: Database Implementation

- Create tables using DDL (CREATE, ALTER, DROP, TRUNCATE)
- Apply constraints (PK, FK, CHECK, UNIQUE, NOT NULL)
- Perform DML operations (INSERT, UPDATE, DELETE)
- Write queries using SELECT, WHERE, GROUP BY, HAVING
- Implement joins (INNER, LEFT, RIGHT, FULL)
- Implement subqueries
- Create views
- Create indexes

Deliverables:

- SQL scripts
 - Query outputs
-

Sprint 3: Advanced SQL and Optimization

- Implement functions (scalar and table-valued)
- Create triggers (AFTER INSERT, UPDATE, DELETE)
- Implement transaction control (COMMIT, ROLLBACK)
- Apply DCL commands (GRANT, REVOKE)
- Validate performance using indexes

Deliverables:

- Trigger scripts
 - Function scripts
 - Transaction demo
-

Final Demonstration

- End-to-end data flow execution
 - Complex queries using joins and subqueries
 - Trigger execution demonstration
 - Transaction rollback scenario
 - Access control simulation
-

GitHub Submission Guidelines

All teams must submit the project using GitHub.

Repository Structure:

```
ABC-Domain-DBMS-Project/  
├── README.md  
├── ERD/  
├── SQL/  
├── Documentation/  
└── Output/
```

README must include:

- Project description
 - Features implemented
 - Steps to execute scripts
 - Sample queries and outputs
-

Domain-Specific Problem Statements

P1 – Hospital Management System

Problem Statement

Design a database system for a hospital to manage patient records, doctor details, appointments, treatments, and billing information.

The system should enable:

- Patient registration and medical history tracking
- Doctor scheduling and specialization mapping
- Appointment booking and status tracking
- Treatment and prescription records
- Billing and payment tracking

The solution must ensure data integrity, support complex queries for reporting (e.g., patient history, doctor workload), and automate updates using triggers.

P2 – Bank Management System

Problem Statement

Design a database system for a banking institution to manage customers, accounts, and financial transactions.

The system should enable:

- Customer onboarding and profile management
- Account management (Savings, Current)
- Fund transfers and transaction tracking
- Balance updates and validations
- Audit logging of transactions

The database must support secure data handling, transaction consistency using TCL commands, and efficient querying for reports such as account summaries and transaction history.

P3 – Blood Bank Management System

Problem Statement

Design a database system to manage blood donors, blood inventory, and request handling.

The system should enable:

- Donor registration and eligibility tracking
- Blood group inventory management
- Blood request and allocation
- Donation history tracking
- Expiry and availability monitoring

The system must ensure accurate inventory updates, enforce constraints, and provide reporting on stock levels and donor activity.

P4 – Hotel Reservation System

Problem Statement

Design a database system for managing hotel operations including room booking, customer management, and billing.

The system should enable:

- Customer check-in and check-out
- Room availability tracking
- Reservation management
- Billing and payment processing
- Room category and pricing management

The database must support real-time booking updates, prevent double booking, and generate reports such as occupancy rates and revenue.

P5 – Event Management System

Problem Statement

Design a database system to manage events, participants, bookings, and payments.

The system should enable:

- Event creation and scheduling
- Participant registration
- Ticket booking and allocation
- Payment tracking
- Event capacity management

The solution must support querying for attendee lists, event performance, and automate updates such as seat availability.

P6 – Real Estate Property Management System

Problem Statement

Design a database system to manage property listings, buyers, sellers, and transactions.

The system should enable:

- Property listing and categorization
- Buyer and seller management
- Property booking and sales tracking
- Pricing and availability updates
- Transaction history

The system must ensure accurate data relationships and provide reports such as available properties and completed transactions.

P7 – E-Ticket Booking System

Problem Statement

Design a database system for an online ticket booking platform for movies, transport, or events.

The system should enable:

- User registration and login details
- Event/movie scheduling
- Seat availability tracking
- Ticket booking and cancellation
- Payment and booking history

The system must prevent overbooking, ensure transaction consistency, and support queries for booking summaries and availability.

Overview

This capstone project simulates a **real-world enterprise database system** for a given domain (e.g., Banking, Healthcare, Hospitality, Events, Real Estate, Ticketing, etc.).

Learners will design and implement a **fully normalized relational database** using Microsoft SQL Server, applying core concepts such as DDL, DML, DCL, TCL, constraints, joins, subqueries, views, indexes, and triggers.

The project follows an **Agile-based execution model** to ensure iterative development, validation, and real-world readiness.

Project Objective

To design and develop a **scalable, normalized, and secure database system** that supports domain-specific operations such as:

- Entity management (users, customers, assets, etc.)
 - Transaction handling (bookings, payments, operations)
 - Data integrity enforcement using constraints
 - Query optimization using indexes and joins
 - Automation using triggers
 - Controlled access using DCL commands
-

Functional Requirements

Core Modules (Domain-Agnostic)

Each project (P1-P7) must include:

1. **Master Data Management**
 - Store core entities (e.g., Customer, Patient, User, Property, Event, Ticket)

2. **Transaction Management**
 - Handle domain-specific operations (e.g., booking, transfer, reservation, allocation)
 3. **Relationship Mapping**
 - Define relationships using PK-FK constraints
 4. **Data Integrity & Validation**
 - Use NOT NULL, CHECK, UNIQUE constraints
 5. **Querying & Reporting**
 - Generate reports using joins, subqueries, aggregate functions
 6. **Automation**
 - Implement triggers for audit/logging
 7. **Performance Optimization**
 - Use indexes for faster query execution
 8. **Access Control**
 - Simulate role-based access using GRANT and REVOKE
-

Technology Stack

- Database: Microsoft SQL Server
 - Query Language: SQL (T-SQL)
 - Tools: SQL Server Management Studio (SSMS)
 - Version Control: GitHub
 - ERD Tool: Draw.io / Visual Paradigm
-

Sprint-Based Execution Plan

Sprint 1: Database Design & Normalization

Tasks

- Identify entities and attributes (no predefined structure)
- Apply normalization up to **3NF/BCNF**
- Define relationships (1-1, 1-M, M-M)
- Prepare ER Diagram

Deliverables

- ER Diagram
 - Normalization Document
 - Entity-Relationship Justification
-

Sprint 2: Database Implementation

Tasks

DDL Operations

- CREATE, ALTER, DROP, TRUNCATE

Table Design

- Define:
 - Primary Keys
 - Foreign Keys
 - Constraints (CHECK, UNIQUE, NOT NULL)

DML Operations

- INSERT, UPDATE, DELETE

Querying

- SELECT with:
 - WHERE, GROUP BY, HAVING
 - ORDER BY

Joins

- INNER, LEFT, RIGHT, FULL

Subqueries

- Scalar, correlated, nested

Views

- Simple views
- Join-based views
- Aggregate views

Indexes

- Create clustered/non-clustered indexes
-

Deliverables

- SQL scripts for tables & data
 - Views and queries
 - Sample outputs
-

Sprint 3: Advanced SQL + Optimization

Tasks

Functions

- Scalar functions
- Table-valued functions

Triggers

- AFTER triggers for:
 - Insert
 - Update
 - Delete

Transactions (TCL)

- COMMIT
- ROLLBACK

DCL

- GRANT
- REVOKE

Performance

- Index usage validation
-

Deliverables

- Trigger scripts
 - Function scripts
 - Transaction handling demo
-

Final Demonstration Requirements

Each team must demonstrate:

- End-to-end flow (e.g., create → update → query → delete)
 - Join-based reporting
 - Trigger execution
 - Transaction rollback scenario
 - Role-based access simulation
-

Evaluation Criteria

Area	Weightage
Database Design (ERD + Normalization)	20%
SQL Implementation (DDL + DML)	25%
Querying (Joins, Subqueries, Views)	20%
Advanced Features (Triggers, Functions, Indexes)	20%
GitHub + Documentation	15%

GitHub Submission Guidelines

All teams must submit the project via GitHub repository.

Repository Structure

None

```
ABC-Domain-DBMS-Project/
|— README.md
|— ERD/
|   └─ er_diagram.png
|— SQL/
|   └─ ddl_scripts.sql
|   └─ dml_scripts.sql
|   └─ queries.sql
|   └─ views.sql
|   └─ triggers.sql
|   └─ functions.sql
|— Documentation/
|   └─ normalization_report.pdf
|— Output/
|   └─ sample_results.xlsx
```

README Must Include

- Project Title

- Domain Description
 - Features Implemented
 - Database Design Approach
 - How to Execute Scripts
 - Sample Queries & Outputs
-

Submission Checklist

- ✓ ER Diagram uploaded
- ✓ Normalization explained
- ✓ SQL scripts executable
- ✓ Minimum 10 complex queries
- ✓ At least 1 trigger + 1 function
- ✓ Proper folder structure
- ✓ README completed